ReBAC V3.0 — Identity Graph & Subject Abstraction

> ***Branch:*** `ReBAC-V3` | ***Builds on:*** *V2.3 Compiler Intelligence*

The Problem V2.3 Left Unsolved

In all previous versions, the left side of every relationship is always a **User**:

```
userId=1 ──[doctor_of]──> patient:101
```

But in real organizations, permissions are assigned to **groups**, **teams**, **departments**, and **service accounts** — not directly to individual users. A hospital grants the "Doctors" group access to all ICU patients, not each doctor individually.

The broken scenario in V2.3:

```
Dr. Raj ──[doctor_of]──> Patient101  (direct, works)
Dr. Meena ──[doctor_of]──> Patient101  (direct, works)
Dr. Ravi ──[doctor_of]──> Patient101  (direct, works)
...×200 doctors — 200 rows per patient, hundreds of thousands of
tuples
```

The correct model (V3.0):

```
Raj ──[member]──> Doctors (group)
Doctors ──[doctor_of]──> Patient101

checkPermission(Raj, Patient101, "view")
→ Resolve: Raj ∈ Doctors
→ Check: Doctors has doctor_of on Patient101
→ ALLOW
```

This is the **Identity Graph** — a separate graph where subjects (users, groups, teams) are nodes and `member` edges encode membership.

V3.0 Architecture

Dual Graph Model

```
Identity Graph                    Resource Graph
──────────────                    ──────────────

  Raj                               Patient101
   │ member                              ↑
   ▼                                 [doctor_of]
 Doctors                            Doctors
   │ member                         (subject)
   ▼
 MedicalStaff
   │ member
   ▼
 HospitalStaff
```

Two graphs, one authorization check:

1. **Identity graph traversal** — BFS/DFS from User → expand all reachable group memberships → `subjectSet: Set<number>`
2. **Resource graph lookup** — check if ANY subject in `subjectSet` has the required relation on the target resource

SubjectResolver — Membership Expansion

```
// One BFS traversal per user per request
const subjectSet: Set<number> = await
SubjectResolver.resolveSubjectSet(userId);
// subjectSet = {userId=1, groupId=5 (Doctors), groupId=8
(MedicalStaff), groupId=12 (HospitalStaff)}
```

Then the direct lookup becomes:

```
SELECT 1 FROM Relationship
WHERE relation = 'doctor_of'
  AND objectId = 101
  AND subjectId IN (1, 5, 8, 12)   -- any member in the chain
```

Subject Type Hierarchy

```
Subject (abstract)
 ├── User
 ├── Group
 ├── Team
 ├── Department
 ├── ServiceAccount
 └── ExternalIdentity
```

DSL declaration:

```
subject user
subject group
subject team
```

Critical Review

✅ What V3.0 Gets Right

1. **Subject abstraction is the correct generalization.** This is identical to Zanzibar's `user` type becoming `subject` in OpenFGA's model — the subject can be any entity that can hold relationships.
2. **Two-graph model** (identity graph + resource graph) correctly separates *who you are* from *what you can access*.
3. **SubjectResolver pre-computation** is a smart optimization — resolve memberships once per request, not once per rule evaluation.

⚠️ Critical Missing: Zanzibar's `#relation` Notation

Zanzibar's full tuple syntax is:

```
(user:raj#member, viewer, document:doc1)
```

The `#member` suffix on the subject means "the set of users who are members of raj" — this is the **subject set** concept.

V3.0 models this implicitly through `SubjectResolver`, but the DSL cannot express it explicitly:

```
# OpenFGA style — what V3.0 cannot express yet:
relation viewer: user | group#member | team#member
```

V3.0 only supports `subject user` and `subject group` — it cannot express "the `member` set of a group" as a typed relation on a resource.

⚠️ Performance Concern: N+1 Membership Queries

`SubjectResolver.resolveSubjectSet()` performs BFS through the identity graph. For deep nesting:

```
Raj → Doctors → MedicalStaff → HospitalStaff → AllEmployees
```

This is 4 BFS levels, each requiring a database query. At scale, this should be **pre-materialized** (cached) rather than computed on every request.

SpiceDB solves this with **dispatch caching** — membership graphs are cached and invalidated lazily.

❌ Incorrect: Flat `member` Relation

V3.0 hardcodes `member` as the group membership relation. But organizations use many membership relation types:

- `member` — direct group member
- `owner` — group admin
- `manager` — reports to this person's group

The identity graph should support **typed membership**, not just `member`.

Comparison with Identity Models

Concept	V3.0	Zanzibar	SpiceDB	OpenFGA
Subject types	✅	✅	✅ full	✅ full
Nested groups	✅	✅	✅	✅
`#relation` notation	❌	✅	✅	✅
Typed membership	❌ partial	✅	✅	✅
Service accounts	✅	✅	✅	✅
Membership caching	❌	✅	✅	✅

Summary

V3.0 is a critical architectural leap — it introduces the **Identity Graph**, making the authorization engine suitable for organizations where permissions belong to groups and teams, not just individuals. The dual-graph traversal model and SubjectResolver design are correct. The main gaps are: the lack of Zanzibar's `#relation` notation for subject sets, typed membership relations, and membership result caching.